

A Governed Harness for Auditable LLM-Driven ML Experimentation

[Author names omitted for review]

[Institution omitted for review]

[City omitted for review], [Country omitted for review]

[omitted]

ABSTRACT

Autonomous ML experimentation agents can propose code edits, execute training runs, and select follow-up trials, but task metrics alone do not reveal whether those actions were bounded, auditable, failure-aware, or reproducible. We argue that governed autonomous experimentation requires a complementary evaluation layer: a suite of *governance metrics*—acceptance rate, invalid-action rate, repeated-bad rate, provenance completeness, artifact capture completeness, and a failure taxonomy—that characterise the reliability of the experimentation process, not just its output. We present a governed harness that operationalises this evaluation framework. In the Agent = Model + Harness decomposition [1], our harness owns trial lifecycle, scope enforcement, and audit; the LLM manager is a replaceable component. The framework separates proposal, execution, validation, and decision authority; enforces editable-scope constraints via a node specification; records append-only trial ledgers with complete provenance; and exports the full governance metric suite per campaign. We validate the framework on the ResNet-trigger ML benchmark with a five-trial real-worker campaign (complete provenance for all trials, documented failure taxonomy, 100% artifact completeness) and a memory ablation across two manager backends—one deterministic, one stochastic LLM-driven. Task-metric AUC improves modestly (+0.000933) as secondary evidence that the loop runs correctly; the central result is that lifecycle decisions, failures, and provenance are jointly auditable for every trial.

KEYWORDS

agentic AI, autonomous experimentation, harness engineering, governed control plane, audit, reproducibility, memory ablation

ACM Reference Format:

[Author names omitted for review]. 2026. A Governed Harness for Auditable LLM-Driven ML Experimentation. In *Proceedings of KDD Workshop on Agentic AI Evaluation (KDD AAE'26)*. ACM, New York, NY, USA, 11 pages.

1 INTRODUCTION

Autonomous ML experimentation agents can now propose code edits, run training jobs, inspect metrics, and choose follow-up trials. Evaluating such systems by final task score alone is insufficient. A

higher validation AUC does not tell us whether the agent edited only allowed files, whether failed trials were preserved, whether no-op patches were rejected, whether keep/discard decisions were made by an external verifier, or whether the system repeatedly attempted known-bad ideas. For scientific ML, these process properties are not implementation details; they determine whether an autonomous experiment is auditable.

This paper argues that autonomous ML experimentation should be evaluated as a governed process. In the Agent = Model + Harness decomposition, the model proposes actions, but the harness supplies tools, memory, execution boundaries, state tracking, and verification. We instantiate that idea as a control plane for ML experimentation: a manager proposes one bounded change, a worker materialises the patch and run artifacts, and the control plane owns lifecycle, scope validation, metric parsing, keep/discard decisions, memory updates, and append-only records.

The central contribution is a governance-metric evaluation protocol for autonomous experimentation. We report acceptance rate, invalid-action rate, repeated-bad rate, provenance completeness, artifact capture completeness, and a failure taxonomy alongside the task metric. These metrics expose behaviours that task score hides: invalid edit attempts, missing metrics, repeated degraded proposals, no-op patches, stale pending trials, and unauditable decisions. A campaign with modest task improvement but complete lifecycle records is more scientifically interpretable than a campaign with an unexplained score increase.

We implement the protocol in `autoresearch_harness`, a local harness around a controlled ResNet-trigger waveform-classification node. The node specification declares the editable surface (`train.py`), frozen data and pipeline files, the training command, metric parser, acceptance rule, and validity checks. The control plane then enforces a fixed-budget trial lifecycle: proposal, pending guard, patch/run, scope and no-op validation, metric parsing, deterministic decision, append-only ledger write, memory update, and guard release.

Empirically, we use the harness to run a five-trial real-worker campaign and optional strengthening runs on the same node. The primary campaign records two kept valid edits and three failed-invalid trials with complete provenance. Longer 10-trial memory runs show full lifecycle coverage and a clearer memory signal: the no-memory arm has repeated-bad rate 0.60, while both memory arms reduce it to 0.40 and reach a higher best validation AUC. A five-seed baseline replicate shows substantial task-metric variation, with mean validation AUC 0.784755 and bootstrap 95% CI [0.774809, 0.798329], so we treat AUC as secondary evidence rather than the headline claim.

This framing leads to three contributions:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

KDD AAE'26, August 2026, Barcelona, Spain

© 2026 Association for Computing Machinery.

- a governed control-plane protocol that separates proposal authority from execution, validation, and keep/discard authority;
- an append-only audit schema and artifact pipeline that records proposal, patch, run log, parsed metric, decision, reproducibility hashes, and failure category for every trial;
- an evaluation methodology for autonomous ML experimentation based on governance metrics, stress failures, memory ablation, and seed/noise guardrails.

The rest of the paper is organised as follows. Section 2 positions the work relative to autonomous code-space exploration, scientific reproduction systems, harness engineering, experiment tracking, and AutoML. Section 3 describes the governed control plane. Section 4 defines the evaluation protocol and governance metrics. Section 5 reports campaign, ablation, stress, and reproducibility results. Section 6 discusses limitations and the path to broader multi-node evaluation.

2 BACKGROUND AND RELATED WORK

Autonomous experimentation sits at the intersection of coding agents, AutoML, experiment tracking, and harness engineering. The key distinction in this paper is that we evaluate the harness that governs the agent loop, not only the model that proposes code edits. In Trivedy’s Agent = Model + Harness decomposition, the control plane, ledger, scope validator, memory interface, metric parser, and state machine are the harness components that make an autonomous experiment bounded and auditable.

2.1 Autonomous Code-Space Exploration

AIDE (Jiang et al. 2025) is the closest related academic system. Both AIDE and autoresearch_harness use LLMs to propose atomic code changes that are evaluated by a hard-coded metric. The structural difference is where lifecycle authority lives: AIDE couples search policy, evaluator, and summarisation inside one optimisation loop, while this work separates proposal generation from governance.

Table 5: Comparison with AIDE.

AIDE optimises the ML metric; we govern the experimentation process. These are complementary goals. An AIDE-style tree-search manager could be plugged into this harness, but it still should not directly commit trial state.

2.2 Reproducibility and Auditing Pipelines

SHARP (Siegel et al. 2025) is the most relevant system on human-agent collaboration for scientific reproducibility. SHARP uses an agent to reproduce prior ML experiments by re-executing documented procedures, compare outputs against original claims, and surface discrepancies. It is stronger on external validation and full pipeline reproducibility for published work.

This work is complementary but differently positioned. SHARP reproduces existing experiments; autoresearch_harness governs autonomous generation of new ones. SHARP’s contribution is a reproducibility-checking pipeline; this paper’s contribution is a governance and evaluation protocol for the agent loop that runs while experiments are being created. SHARP does not define trial lifecycle states, append-only audit ledgers, scope enforcement, failure taxonomies, or memory ablation protocols. The main gap relative

to SHARP is not concept design but clean empirical evidence under a governed protocol.

Other open-source agent systems also inform the design. ml-intern motivates event traces, local-model support, headless operation, and optional interactive handoff. multiautoresearch and deep-research-style systems motivate multi-track project organisation and accumulated learnings. Hermes-style systems motivate durable routines and trajectory compression. These systems are useful implementation references, but they do not define an evaluation protocol for governed autonomous experimentation.

Mind2Web (Deng et al. 2023) represents a different evaluation philosophy: broad benchmark-driven evaluation across many web domains. Unlike benchmark-driven evaluation across many domains, we prioritise a single real task node with complete provenance. This trades breadth for inspectability: every proposal, patch, run log, parsed metric, and keep/discard decision can be audited.

2.3 Harness Engineering

The harness engineering literature independently validates this framing. Designing the environments, feedback loops, and control systems is now recognized as the primary engineering challenge for agentic AI. Trivedy (2026) frames the harness as everything around the model: tools, sandboxes, memory, middleware, and execution hooks. Böckeler (2026) argues that trust in agent output requires constraining the solution space with guides and sensors. OpenAI (2026) describes production agent work as an observability and feedback-loop problem: give the agent a map, not a manual, and turn failures into enforceable harness improvements. Anthropic (Young 2025; Rajasekaran 2026) similarly emphasises progress tracking, verifier separation, and skeptical evaluation for long-running agents.

This paper instantiates those ideas in autonomous ML experimentation. The node specification is a computational guide; the metric parser, pending-trial guard, and state machine are computational sensors; the manager prompt and memory context are inferential guides; the repeated-bad detector is an inferential sensor. The key design choice is that inferential components may propose and summarise, but deterministic components own validity and lifecycle transitions.

2.4 Experiment Tracking and AutoML

Experiment trackers such as MLflow and Weights & Biases record outcomes, metrics, and artifacts after a run exists. They do not govern the agent loop, enforce bounded execution before a patch is applied, or decide whether invalid trials should become first-class audit objects.

AutoML systems, including hyperparameter optimisation and neural architecture search, optimise within a predefined search space. They are not designed to audit an agent’s self-directed exploration, classify invalid edit attempts, or explain why a proposed code change was kept, discarded, or failed invalid.

None of the above provides the full governed control plane: scope enforcement, append-only audit ledger, failure taxonomy, and memory ablation for evaluating autonomous experimentation.

Dimension	AIDE	autoresearch_harness
LLM role	Proposes code and selects the next search node	Proposes a bounded experimental change only
Control plane	Hard-coded tree search plus evaluator	Governed lifecycle state machine
Keep/discard owner	Hard-coded metric function $h(s)$ inside the search loop	Control plane with deterministic, auditable decision rule
Scope enforcement	No explicit editable-path whitelist	NodeSpec editable-path whitelist and scope validator
Audit ledger	No append-only trial ledger	Append-only JSONL ledger with provenance IDs
Failure taxonomy	Not a first-class artifact	Six-category experimentation failure taxonomy
Memory/context	Tree summarisation operator $\Sigma(T)$	Three-mode memory ablation: none, summary, summary with rationale
Reproducibility claim	Metric performance across code-space search	Full provenance from proposal to patch, run log, metric, and decision

Dimension	SHARP	autoresearch_harness
Primary goal	Reproduce and verify prior published experiments	Govern and audit autonomous generation of new experiments
Control plane	Agent executes a documented procedure	Governed state machine owns trial lifecycle
Decision authority	Human reviewer inspects agent output	Control plane decides keep/discard/failed-invalid deterministically
Audit trail	Execution log of reproduction attempt	Append-only JSONL ledger with provenance IDs per trial
Failure handling	Discrepancy reported to human	First-class failed_invalid records with failure categories
Scope enforcement	Procedure document bounds the agent	NodeSpec editable-path whitelist enforced before patch is applied

3 GOVERNED CONTROL PLANE

The system is a governed harness for autonomous ML experimentation. The manager proposes an experimental change; the worker materialises a patch and run artifacts; the control plane validates scope, executes the lifecycle, parses metrics, decides keep/discard/failed-invalid, updates memory, and appends the authoritative ledger record. Managers and workers are replaceable, but managers cannot commit trial state.

3.1 Manager/Worker Separation and Replacement Principle

The manager is responsible for proposal generation: a summary, rationale, target file, and intended experimental effect. The worker is responsible for bounded execution: turning that proposal into a patch, running the configured command, and returning artifact references. The control plane is responsible for authority: validity, state transitions, budget accounting, memory updates, provenance IDs, and final decision.

This separation is the replacement principle. A baseline heuristic manager, prompt manager, LangGraph manager, AIDE-style tree-search manager, local worker, Claw-style worker, or future cloud worker can be swapped in without changing the paper-facing governance contract. The append-only ledger remains the source of truth.

Worker replication paths. Two worker implementations cover the full reproducibility spectrum:

- **LocalWorker** (`local_worker.py`): parses “Change PARAM from X to Y” directives from proposal text and applies edits directly with no external dependencies. Suitable for protocol verification without a running LLM or coding agent.
- **ClawWorker** (`claw_worker.py`): generates a packet JSON and invokes the legacy claw-harness loop, which requires a live AI coding agent (such as Claude Code) for free-form LLM proposals. For structured proposals, ClawWorker routes through a deterministic constant-patch path that requires only Ollama, not a coding agent.

Both workers implement the same Worker protocol and produce ledger records under the same schema; the control plane and paper-facing governance contract are identical regardless of which worker is used.

3.2 Trial Lifecycle

Each trial follows a fixed lifecycle:

```

flowchart LR
    A["Budget slot opened"] --> B["Manager proposes change"]
    B --> C["Pending guard written"]
    C --> D["Worker produces patch and log"]
    D --> E["Scope and no-op validation"]
    E --> F["Training command and metric parser"]
    F --> G["Control-plane decision"]
    G --> H["Append-only ledger record"]
    H --> I["Memory update and guard removal"]

```

In pseudocode:

```

for budget_index in campaign_budget:
    proposal = manager.propose(memory_context, node_spec)

```

```

write_pending_guard(campaign_id, trial_id, proposal)
worker_result = worker.run(proposal, node_spec)
validity = validate_patch_scope_and_effect(worker_result.
    patch_ref, node_spec)
if not validity.ok:
    decision = failed_invalid(validity.failure_category)
else:
    metrics = parse_metric(worker_result.raw_log_ref)
    decision = keep_if_metric_improves(metrics,
        current_best)
append_trial_record(decision, provenance, artifacts)
update_memory(decision)
remove_pending_guard()

```

3.3 Scope Enforcement

The ResNet-trigger NodeSpec declares `train.py` as the only editable file. Frozen files include `prepare.py`, `program.md`, node dependencies, data files, and artifact directories. The scope validator checks the generated patch before the candidate can be accepted. A patch touching a frozen path is marked `failed_invalid / invalid_edit_scope`, written to the ledger, and prevented from corrupting node state.

3.4 Pending-Trial Guard

Before worker execution, the control plane writes a pending guard under `experiments/ledgers/`. The guard makes mid-trial crashes auditable instead of invisible. Recovery tooling can list, inspect, and resolve stale guards by appending a failed-invalid ledger record before deleting the guard. This preserves the invariant that every opened budget slot has a terminal audit object.

3.5 Append-Only Ledger

The ledger is an append-only JSONL file per campaign. A trial record contains campaign identity, budget index, proposal summary and rationale, patch and raw-log references, parsed metrics, validity status, decision, failure category, provenance IDs, reproducibility hashes, and timing. The ledger is the durable system of record; generated tables and figures are derived artifacts.

3.6 Memory Modes

The memory interface has three modes:

The ablation asks whether rationale-linked memory reduces repeated poor proposals more than raw history alone.

3.7 Decision Authority

We keep the keep/discard decision in the control plane, not delegated to the manager. Rajasekaran et al. (2026) found that LLM self-evaluation is systematically lenient – agents confidently praise mediocre work. A deterministic, externally-owned decision criterion addresses this.

The decision rule is intentionally simple: a valid candidate is kept only if the configured metric improves over the current best in the configured direction. A valid but non-improving candidate is discarded / `degraded_metric`; an invalid candidate is `failed_invalid` with a specific failure category. The manager may explain a proposal, but it cannot declare it successful.

3.8 Guides and Sensors

Table S1: Harness guides and sensors for autonomous ML experimentation.

This table instantiates Böckeler’s guides/sensors taxonomy in the ML experimentation setting.

3.9 Failure Mode Motivation

Table S2: Agent failure modes and harness controls.

The design treats these failures as expected lifecycle outcomes. Failed trials are audit records, not hidden exceptions.

4 EVALUATION PROTOCOL

The evaluation asks whether an autonomous ML experimentation loop is bounded, auditable, failure-aware, reproducible, and behaviorally affected by governance memory. The task metric is still reported, but governance metrics are primary.

4.1 Benchmark Node

The benchmark is the ResNet-trigger node (`resnet_trigger`), a real scientific ML task for near-threshold detector waveform binary classification. The node trains a ResNet-style classifier on signal and noise traces. The editable surface is deliberately narrow: workers may modify only `nodes/ResNet_trigger/train.py`, while data loading, split logic, node metadata, dependency files, and artifacts are frozen.

The scientific task metric is validation AUC, exposed to the control plane as `val_auc` with `metric_direction=maximize`. The node also prints a compatibility scalar `val_bpb = 1 - best_val_auc` for older worker paths, but the paper reports AUC directly.

4.2 Fixed-Budget Campaign Protocol

Each campaign starts from a reset node state and a fixed budget. The canonical five-trial campaign uses:

For every trial, the control plane records the proposal, patch reference, raw log reference, parsed metrics, validity status, failure category if any, decision rationale, provenance IDs, and reproducibility hashes. A valid improving trial is kept; a valid non-improving trial is discarded; an invalid trial is failed invalid.

4.3 Memory Ablation Design

The memory ablation runs equal budgets under three conditions: `none`, `append_only_summary`, and `append_only_summary_with_rationale`. Each arm uses the same node, budget, metric parser, acceptance rule, editable scope, and starting-state reset. The primary outcome is repeated-bad rate, computed as:

$$\text{RepeatedBadRate} = \text{repeated_bad_proposals} / \text{total_proposals}$$

A repeated-bad proposal is one that repeats the same edit target and edit mechanism as a prior rejected, invalid, or degraded trial without adding a new corrective rationale, constraint, or justification.

The hypothesis is pre-stated before results:

$$\text{repeated_bad_rate}(\text{none}) > \text{repeated_bad_rate}(\text{append_only_summary})$$

Mode	Manager context	
none	No prior-trial context is injected.	
append_only_summary	Prior trial summaries, decisions, metrics, and failure categories are injected.	
append_only_summary_with_rationale	The summary context is augmented with decision rationales, giving the manager targeted failure signals.	

	Guide (feedforward – steers before act)	Sensor (feedback – corrects after act)
Computational (deterministic, fast)	Node spec defining valid scope; editable-path whitelist; budget cap; scope constraints	Metric parser (val_bpb to val_auc); state machine enforcing transition legality; pending-trial guard detecting crashes
Inferential (semantic, LLM-based)	Manager system prompt; memory injection; proposal rationale template	Repeated-bad detector using edit-target and mechanism similarity to flag redundant proposals

Agent failure mode	ML experimentation analogue	Harness fix
Declares victory too early	Manager stops after one apparently good trial	Budget enforcer plus fixed trial count
Leaves broken state	Failed patch leaves repository or node dirty	Pending-trial guard, git before/after checks, reset script
Marks feature done prematurely	Manager claims improvement without parseable metric	FAILED_INVALID state plus metric parser
Does not know how to run the app	Manager proposes an out-of-scope edit	Node spec plus editable-path whitelist

Field	Value
Node	resnet_trigger
Campaign	kdd_main_5trial
Budget	5 trials
Manager	prompt_manager with structured state-aware hyperparameter proposals
Memory mode	append_only_summary_with_rationale
Worker	claw_style_worker with deterministic patch path for constant edits
Acceptance rule	keep iff candidate val_auc improves over current best

```
> repeated_bad_rate(
    append_only_summary_with_rationale
)
```

Rationale-augmented memory should reduce repeated-bad rate more than raw summaries because it gives the manager a targeted failure signal rather than history noise. This hypothesis is grounded in OpenAI’s map-not-manual framing and Böckeler’s combination of feedforward guides with feedback sensors. If the result is flat or reversed, it is a valid negative finding and is reported as such.

Two ablation configurations are reported. The first uses `prompt_manager`, which delegates to `select_structured_hyperparameter_edit`, a deterministic round-robin indexed on `budget_index`. Memory context is injected correctly and differs in content across modes, but the selector’s determinism means it cannot produce a different proposal sequence regardless of memory content. This arm is reported as a design-constrained negative finding.

The second ablation uses `LangGraphManager` (`langgraph_manager`), which calls `qwen2.5-coder:7b` via Ollama at `temperature=0.2`. The full formatted memory context is passed as part of the prompt and a free-form JSON proposal is parsed. This is a genuinely stochastic backend where memory context can in principle influence proposal selection. Campaign IDs: `lg_ablation_none`, `lg_ablation_append_only_summary`, `lg_ablation_append_only_summary_with_rationale`. Memory injection is verified per-trial by recording the SHA-256 of the formatted context in the ledger `extra` field; all 15 context hashes are distinct,

confirming that memory accumulates correctly across trials and differs across modes.

A secondary failure mode emerges in the LangGraph ablation: `edit_failed`. The `claw_style_worker` requires a live AI coding agent (such as Claude Code or a claw-harness agent) to translate free-form LLM proposals into file patches. Without an agent present, training executes against the unmodified source and the legacy loop reports “worker did not modify train.py.” The control plane correctly classifies these trials as `failed_invalid` / `edit_failed` — it does not accept a baseline metric obtained on an unedited file as a keep. This demonstrates the governance layer’s integrity under a new failure mode not present in the main campaign.

4.4 Stress Trial

The stress trial exercises invalid edit-scope handling. A dedicated scope-violation worker generates a patch that touches a forbidden file. The expected control-plane response is:

```
worker attempts forbidden edit
-> scope validator rejects patch
-> trial is marked failed_invalid / invalid_edit_scope
-> patch and raw log references are retained
-> git state remains unchanged
-> pending guard is removed
```

This trial demonstrates that invalid actions are first-class audit objects rather than missing data.

4.5 Governance Metrics

Governance metrics are the primary evaluation criterion in this work. They measure the reliability of the experimentation process itself — how many trials were valid, how many failures were classified and recorded, whether provenance is complete, and whether the agent repeated known-bad proposals. A campaign can show a high task metric while simultaneously hiding failures, producing unauditable decisions, or running on corrupted state; governance metrics make these behaviors visible. Task-metric AUC is reported as secondary evidence that the loop runs correctly end-to-end.

Definition 1 (Governance Metrics Suite). Given a fixed-budget campaign of N trials, the following metrics characterise process reliability:

A well-governed campaign is not necessarily one with high acceptance rate. A campaign with 60% invalid rate but 100% provenance completeness, correct failure taxonomy, and no scope violations provides more interpretable evidence than one with 100% acceptance rate and no audit trail.

4.6 Failure Taxonomy

Table 2: Failure taxonomy for autonomous ML experimentation.

The taxonomy is included in the main experiment section so the reader can interpret failures before seeing results.

5 RESULTS

Results are presented in governance-first order. The main campaign (kdd_main_5trial) uses a real `claw_style_worker` with `prompt_manager` on the ResNet-trigger node. All governance and task-metric numbers below are derived from the current ledgers in `experiments/ledgers/`. The full artifact suite is indexed in `artifact_manifest.json`.

5.1 Main Campaign Governance

Table 1: Main campaign governance and optimisation summary.

Per-trial summary:

The campaign demonstrates lifecycle diversity: two valid improving edits are kept with full patch and provenance artifacts, and three runtime-invalid trials are classified and recorded as first-class audit objects. Provenance is complete for all five records. The campaign does not include a discarded valid-but-worse trial; full three-way lifecycle diversity would require a longer campaign or a valid non-improving trial.

5.2 Failure Taxonomy Results

Table 2 result artifact: `paper/tables/failure_taxonomy.csv`.

The main campaign’s runtime errors represent proposals that changed source text but caused the training command to exit nonzero. The LangGraph ablation campaigns surface a distinct new failure mode: `edit_failed`, detailed below. The stress campaigns each exercise one forced failure path at 100% rate.

5.3 Memory Ablation

Two ablation configurations are reported, differing in the manager backend.

5.3.1 Arm A: `prompt_manager` (deterministic round-robin). Three arms at equal budgets (5 trials each), differing only in the memory context injected into the proposal selector. Campaign IDs: `ablation_none`, `ablation_append_only_summary`, `ablation_append_only_summary_wi`.

Result: flat. All three arms produced identical proposal sequences, outcomes, and final metrics (2 kept, 0 discarded, 3 failed-invalid, `repeated_bad_rate` = 0.40, `best_val_auc` = 0.774711). The mechanistic explanation is in Section 4: the deterministic round-robin makes memory-based avoidance structurally redundant. This is a design-constrained negative finding — the memory subsystem and repeated-bad detector function correctly, but the proposal generator is insensitive to memory content.

5.3.2 Arm A2: 10-trial `prompt_manager` extension. After the five-trial run, we ran an optional 10-trial extension under the same node, manager, worker, metric parser, and reset discipline. The longer budget exposes behaviour hidden by the shorter run.

The 10-trial extension shows a memory effect at the process level: both memory modes reduce repeated-bad rate from 0.60 to 0.40 and reach a higher best validation AUC than the no-memory arm. The two memory modes are tied, so the stronger pre-stated ordering is still not confirmed; specifically, rationale memory does not outperform summary-only memory in this run. The single failed-invalid trial in each memory arm is `proposal_precondition_failed`, meaning the state-aware structured selector could not find a remaining non-no-op effective edit at the current state. Evidence is recorded in `paper/tables/p8_memory10_summary.csv`.

5.3.3 Arm B: `langgraph_manager` (stochastic LLM, qwen2.5-coder:7b, temperature=0.2). Three arms using the same node, budget (5), metric parser, acceptance rule, and starting-state reset. Campaign IDs: `lg_ablation_none`, `lg_ablation_append_only_summary`, `lg_ablation_append_on`. Memory injection is verified per-trial via SHA-256 of the formatted context recorded in each ledger record; all 15 context hashes are distinct, confirming that memory accumulates correctly and differs across modes.

Figure 2: `paper/figures/fig2_repeated_bad_rate.svg` plots repeated-bad rate by memory mode (LangGraph arms).

Table 3: LangGraph memory ablation summary.

Proposal diversity result. Despite identical repeated-bad rates, the stochastic backend does show a weak memory effect at the proposal level: the `append_only_summary` arm produced increase-learning-rate at T3 (a divergent proposal class) before reverting to batch-size at T4. The `append_only_summary_with_rationale` arm produced increase-batch-size at all five trials. The none arm produced increase-batch-size at T1–T4 and a minor variant at T5. The pre-stated hypothesis (`none` > `summary` > `rationale`) is not confirmed: `repeated_bad_rate` = 0.80 for all three arms.

edit_failed failure mode. The `claw_style_worker` translates proposals by invoking the `claw-harness` legacy loop, which in turn requires a live AI coding agent (such as Claude Code) to apply free-form LLM proposals as file patches. Without an agent present in this experimental setup, training executed against the unmodified

Metric	Formal definition	What it detects
Acceptance rate	$ kept /N$	Whether the agent produces valid improving edits at all.
Invalid rate	$ failed_invalid /N$	Proportion of trials that could not be evaluated.
Repeated-bad rate	Repeated-bad proposals $/N$	Whether the agent avoids redundant failure modes across trials.
Complete provenance rate	Trials with all five IDs (proposal, patch, run, metric, decision) $/N$	Whether every trial is independently reproducible.
Artifact capture completeness	Trials with captured patch and log refs $/N$	Whether physical evidence is retained for every trial.
Failure taxonomy	Counts per FailureCategory	Whether failures are classified, not silently discarded.
Scope-violation count	Trials rejected for editing frozen files	Whether the agent respects editable-scope constraints.
Gain per budget unit	Net task-metric gain $/N$	Secondary: whether governance overhead impedes improvement.

Category	Definition	Control-plane response
invalid_edit_scope	Patch touched a disallowed file or region.	Mark failed_invalid, retain patch reference, do not commit state.
syntax_error	Code cannot be parsed or imported.	Mark failed_invalid, retain logs, do not keep patch.
runtime_error	Training command exits nonzero.	Mark failed_invalid, retain raw log.
metric_missing	Run completes but expected metric cannot be parsed.	Mark failed_invalid, retain run artifact.
degraded_metric	Valid run is worse than current best.	Mark discarded, revert or avoid committing candidate state.
no_op_patch	Worker produced no effective code change.	Mark failed_invalid, skip training execution.
edit_failed	Worker ran but did not modify the target file (e.g., no AI coding agent present to apply a free-form LLM proposal).	Mark failed_invalid, retain run artifact; do not accept baseline metric as a keep.

Field	Value
Table artifact	paper/tables/main_campaign_summary.csv
Governance metrics	paper/tables/governance_metrics.csv
Campaign	kdd_main_5trial
Node	resnet_trigger
Manager	prompt_manager
Worker	claw_style_worker
Memory mode	append_only_summary_with_rationale
Budget	5
Initial val_auc	0.773778
Best / final accepted val_auc	0.774711
Net gain	+0.000933
Kept / discarded / failed-invalid	2 / 0 / 3
Acceptance rate	0.40
Invalid rate	0.60
Complete provenance rate	1.00
Artifact capture completeness	0.40 (patch artifacts present for kept trials only; invalid trials have empty patch refs by design)

train.py; the legacy loop reported “worker did not modify train.py.” The control plane correctly classified all 15 trials as failed_invalid / edit_failed — it did not accept the baseline metric (val_auc = 0.774711, present in the run log) as a keep. This demonstrates that the governance layer enforces correctness even when a downstream component (the AI coding agent) is absent: the system fails safe rather than accepting an unedited run as an improvement.

5.4 Decision Breakdown

Figure 3: paper/figures/fig3_decision_breakdown.svg shows kept, discarded, and failed-invalid counts across the main, ablation, and stress campaigns.

All eight campaigns in the current evidence set produce non-empty ledgers with no pending guard and complete provenance. The decision distribution is not collapsed into a single score; the

	Trial	Proposal	Decision	Failure	val_auc	Patch artifact		
	1	lower-weight-decay	failed_invalid	runtime_error	—	none		
	2	small-dropout	kept	—	0.773778	present (323 B)		
	3	smaller-kernel	failed_invalid	runtime_error	—	none		
	4	lower-grad-clip	failed_invalid	runtime_error	—	none		
	5	larger-batch	kept	—	0.774711	present (340 B)		
Campaign	Failure category			Count	Rate	Control-plane response		
kdd_main_5trial	runtime_error			3	0.60	Mark failed_invalid, retain run artifact reference.		
lg_ablation_*(all 3 arms)	edit_failed			5 each	1.00	Mark failed_invalid; training ran on unmodified file; baseline metric not accepted as keep.		
kdd_stress_scope	invalid_edit_scope			1	1.00	Reject forbidden-file patch; retain audit artifacts; git state unchanged.		
kdd_stress_noop	no_op_patch			1	1.00	Mark failed_invalid, skip training.		
Memory mode	Budget		Kept	Discarded	Failed-invalid	Best val_auc	Repeated-bad rate	
none	10		3	7	0	0.776756	0.60	
append_only_summary	10		4	5	1	0.782733	0.40	
append_only_summary_with_rationale	10		4	5	1	0.782733	0.40	
Memory mode	Manager	Budget	Kept	Discarded	Failed-invalid	Failure category	Repeated-bad rate	Best val_auc
none	langgraph_manager	5	0	0	5	edit_failed	0.80	—
append_only_summary	langgraph_manager	5	0	0	5	edit_failed	0.80	—
append_only_summary_with_rationale	langgraph_manager	5	0	0	5	edit_failed	0.80	—

governance claim is that the harness correctly records, classifies, and exposes all outcomes including novel failure modes.

5.5 Governance in Action: What Breaks Without Controls

The stress campaigns (kdd_stress_scope, kdd_stress_noop) directly demonstrate what the governance layer prevents. Table 5 contrasts the observed governed outcome against the outcome a naive unguarded loop would produce for each failure scenario.

Table 5: Governance controls demonstrated by stress trials.

Each row is a silent failure in a naive loop that the harness makes visible and auditable. The no-op and scope-violation cases are backed by real ledger records from the stress campaigns. The pending-guard case is structural: the invariant is that every opened budget slot must have a terminal audit object, enforced in code before any new campaign can proceed.

This contrast is the answer to the question “why does governance matter?” An agent that maximises AUC by any means can produce any of the above scenarios. The governance layer ensures the paper-facing metrics reflect only trials that were validly executed, correctly scoped, and fully provenance-tracked.

Table 4 result artifact: paper/tables/provenance_chain.csv.

All five trials in kdd_main_5trial have complete provenance: proposal ID, patch ID, run ID, metric ID, and decision ID are present in every record. The artifact completeness checker paper/tables/artifact_completeness_report.txt reports 88/88 checks passed (100.0%) across all evidence campaigns. The manifest in artifact_manifest.json indexes ledgers, tables, figures, and per-trial artifact references.

Scenario	Naive (unguarded) outcome	Governed outcome	Evidence campaign
No-op patch — worker produces a byte-identical change to <code>train.py</code>	Training runs on unmodified file; baseline metric accepted as a keep; spurious improvement recorded.	Training skipped; trial classified <code>failed_invalid / no_op_patch</code> ; ledger records empty patch hash; git state unchanged.	<code>kdd_stress_noop</code>
Out-of-scope edit — worker touches <code>prepare.py</code> (frozen) alongside <code>train.py</code>	Forbidden file modified; node state corrupted; no record of the violation.	Patch rejected before training; trial classified <code>failed_invalid / invalid_edit_scope</code> ; <code>changed_files: ["prepare.py"]</code> recorded in ledger; git state unchanged.	<code>kdd_stress_scope</code>
Stale pending guard — crash or kill between worker call and guard removal	Next campaign launch overwrites in-flight state; previous trial has no terminal ledger record; ledger integrity violated.	<code>PendingTrialError</code> raised on next launch; operator must resolve via <code>recover_pending.py</code> , which appends a <code>failed_invalid</code> record before clearing the guard.	(Recovery tooling; not a campaign ledger.)

5.6 Task Metric Trajectory

Figure 4: `paper/figures/fig4_trajectory.svg` shows the main campaign `val_auc` trajectory.

AUC is secondary evidence that the loop runs correctly end-to-end, not the primary result. The campaign reports `val_auc` increasing from 0.773778 (T2) to 0.774711 (T5), a net gain of +0.000933 over five trials. This margin is small relative to baseline seed variation: five clean baseline runs over seeds 123-127 have mean `val_auc` = 0.784755, population standard deviation 0.013837, and bootstrap 95% CI for the mean [0.774809, 0.798329]. Therefore we do not interpret the primary campaign’s AUC change as evidence of meaningful optimisation. The central result is that lifecycle decisions, failures, provenance, and artifacts are jointly auditable for each trial — whether or not the task metric improves.

6 DISCUSSION AND LIMITATIONS

6.1 What Is Proven

The current artifact suite proves that the control-plane protocol can run a bounded fixed-budget real-worker campaign, classify keep/discard/failed-invalid outcomes, preserve append-only provenance, recover per-trial artifact references, and expose governance metrics as paper tables and figures.

Specifically, the `kdd_main_5trial` real campaign demonstrates that:

- a `claw_style_worker` backed by `prompt_manager` can execute five real ResNet-trigger training runs under governance;
- two valid improving edits are kept with complete patch diffs, parsed metrics, and provenance chains;
- three runtime-invalid trials are classified, recorded, and exposed as first-class audit objects without corrupting node state or leaving a pending guard;
- provenance is complete for all five records (100% rate);
- the append-only ledger correctly reflects the true sequence of events.

The stress campaigns prove that scope violations (`invalid_edit_scope`) and no-op patches (`no_op_patch`) are recorded as first-class audit objects with artifact references and unchanged git state.

6.2 What Is Not Proven

The following claims are explicitly not made:

- **General autonomous scientist capability.** The system governs a fixed-scope hyperparameter search loop on one node. It is not a general scientific discovery engine.
- **Scientific discovery.** The net AUC gain of +0.000933 over five trials on the ResNet-trigger task is secondary evidence of real execution, not a claim of meaningful ML improvement.
- **Universal optimisation algorithm.** The harness controls the experimentation process; the proposal selector is a structured round-robin, not an optimisation method.
- **Backend-specific dependency.** The governance protocol does not depend on a specific coding agent, LLM, or worker implementation. Manager and worker are replaceable without changing the control-plane contract.
- **Broad benchmark coverage.** All real evidence uses the ResNet-trigger node. Results may not generalise to other ML tasks without additional evaluation.
- **Full three-way lifecycle diversity.** The current main campaign includes kept and failed-invalid outcomes but no discarded valid-but-worse trial. A longer campaign or a valid non-improving proposal would complete the lifecycle demonstration.
- **Rationale memory superiority.** The 10-trial extension shows memory modes reducing repeated-bad rate relative to no memory, but summary-only and summary-with-rationale are tied. We do not claim that rationale memory outperforms summary memory in this configuration.

6.3 Memory Ablation: Layered Negative Finding

The pre-stated hypothesis — `repeated_bad_rate(none) > repeated_bad_rate(summary) > repeated_bad_rate(rationale)` — is not confirmed in either of the two ablation configurations that were run.

prompt_manager arm (design-constrained negative). All three arms produce identical outcomes (2 kept, 3 failed-invalid, `repeated_bad_rate` = 0.40, best `val_auc` = 0.774711). The mechanism is in Section 4: the deterministic round-robin makes memory structurally irrelevant. Memory injection and repeated-bad detection work correctly; the limitation is the proposal generator.

10-trial prompt_manager extension (partial positive). Extending the same structured manager to 10 trials changes the conclusion. The no-memory arm records `repeated_bad_rate` = 0.60, while both memory arms record `repeated_bad_rate` = 0.40 and reach a higher best validation AUC. This supports the weaker claim that memory can reduce repeated poor proposals under a longer budget. It does not support the stronger hypothesis that rationale memory beats summary memory, because the two memory arms are tied.

langgraph_manager arm (stochastic LLM, genuinely behavioural). The stochastic backend confirms that memory context is being built up correctly per trial — all 15 SHA-256 context hashes are unique and differ across modes. A weak behavioral signal is present: the `append_only_summary` arm diverged at T3 (increase-learning-rate) before reverting, whereas the `append_only_summary_with_rationale` arm never diversified. Despite this, `repeated_bad_rate` = 0.80 for all three arms. Two factors explain the flat repeated-bad rate: the LLM gravitates to batch-size proposals regardless of prior failure context, and the dominant failure mode is `edit_failed` rather than a semantic repetition. The `edit_failed` result is also a positive governance finding: the control plane correctly refused to keep 15/15 trials where training ran against unmodified code, demonstrating fail-safe behaviour under a novel failure condition.

This is a layered finding rather than a simple win. It correctly characterises separate limits: (1) a five-trial deterministic run was too short and too constrained to expose a memory effect; (2) a 10-trial deterministic run shows memory reducing repeated-bad rate but not a rationale-over-summary advantage; (3) the stochastic LangGraph/free-form-proposal interface requires a live AI coding agent unless proposals are converted into structured edits. The governance infrastructure itself — memory injection, SHA-256 verification, repeated-bad counting, fail-safe classification — functions correctly in all configurations.

6.4 Why Governance Metrics Matter

Final AUC alone cannot answer whether an autonomous experiment was scientifically credible. An agent could improve a metric while repeatedly attempting invalid edits, hiding failed runs, corrupting state, or keeping decisions that cannot be audited. Governance metrics make those behaviors visible.

Acceptance rate, invalid rate, repeated-bad rate, provenance completeness, artifact capture completeness, and failure categories describe the reliability of the experimentation process around the model. This distinction is the main contribution: the harness does not replace task metrics; it makes task metrics interpretable by attaching lifecycle, validity, and provenance context to every trial.

6.5 Limitations

Single node. All real evidence uses the ResNet-trigger node. We cannot rule out that governance behavior reported here is specific to narrow hyperparameter editing on this node. Applying the framework across multiple nodes with holdout splits would strengthen the generalisation claim.

No holdout evaluation node. Better-Harness-style holdout evaluation nodes are not yet implemented. Without holdout nodes, it is possible for the proposal selector to overfit to the evaluation surface. This is future work.

No discarded trial in main campaign. The current five-trial campaign does not include a valid-but-worse trial. Full three-way lifecycle diversity (kept, discarded, failed-invalid) requires a longer campaign or a proposal that produces a valid metric below the current best.

Memory ablation: partial signal. The 10-trial structured-manager extension shows memory reducing repeated-bad rate relative to no memory, but does not distinguish summary memory from rationale memory. The `langgraph_manager` ablation with a real LLM confirms memory injection but shows only a weak behavioral effect: one arm diversified at T3, but `repeated_bad_rate` is 0.80 across all arms. A stronger test of the memory hypothesis would require a proposal generator that more reliably acts on failure context, or a task with a wider editable surface where different memory modes lead to structurally different proposals.

Stress artifacts regenerated from worker code. The stress trial artifact files (`patch.diff`, `run.log`) were missing on disk after a prior reset and were reconstructed from the deterministic worker code (`stress_scope_violation_worker`, `stress_no_op_patch_worker`). The ledger records are original and unchanged. The reconstructed files match what the workers deterministically produce.

Task-metric noise. Five baseline seeds produce validation AUCs from 0.771511 to 0.810711, with mean 0.784755 and bootstrap 95% CI [0.774809, 0.798329]. This variation is larger than the primary five-trial campaign's +0.000933 gain, so task-metric improvement should be treated as secondary evidence that the loop runs, not as an optimisation claim.

Reproducibility and worker dependencies. The `claw_style_worker` has two internal execution paths. The deterministic patch path — gated on `proposal.extra["deterministic_patch"]` and `proposal.extra["structured_edit"]` — applies constant-value edits directly and requires no external coding agent. The legacy loop path translates free-form LLM proposals into patches by invoking a live AI coding agent (Claude Code or a claw-harness agent). The `edit_failed` failure mode (Section 5) arises when the legacy loop path is taken without an agent present: training runs on the unmodified source, and the control plane correctly refuses to keep the result.

The `LangGraphManager` deterministic patch bridge (`_extract_structured_edit`, implemented in the harness) eliminates this dependency for proposals that include `structured_param/old_value/new_value` fields. Any replication attempt using `LangGraphManager` with a model that produces structured proposals will route through the deterministic path without requiring a coding agent.

For fully dependency-free replication — no Ollama, no coding agent — the `LocalWorker` (`src/autoresearch/worker/local_worker.py`) parses “Change PARAM from X to Y” directives from any proposal text and applies them directly. This worker exercises the full control-plane lifecycle, including scope validation, metric parsing, and ledger recording, without external tool dependencies. Reviewers wishing to verify the governance protocol can run a three-trial dry campaign with `LocalWorker` or `DryRunWorker` using only standard Python dependencies.

6.6 Generalisation Path

The NodeSpec YAML pattern generalises the harness to new ML experiments without code changes; each spec is a harness template for a class of experiments. A stronger follow-up evaluation would:

- add multiple node specs, including a holdout node not used for development;
- rerun the memory ablation using LangGraphManager with a live coding agent present to close the edit loop, so that free-form proposals are actually applied and the behavioral effect of memory on valid-trial outcomes can be assessed;
- compare multiple manager backends (baseline, prompt, Lang-Graph) under equivalent budgets and worker modes;
- report bootstrap confidence intervals for governance metrics across repeated seeds.

7 CONCLUSION

We presented a governed harness for autonomous ML experimentation that treats trial lifecycle, validity, provenance, and failure handling as first-class evaluation objects. The central design choice

is separation of authority: managers propose experiments, workers materialise patches and run artifacts, and the control plane owns scope enforcement, metric parsing, keep/discard decisions, memory updates, and append-only records.

On the ResNet-trigger node, the harness produces auditable real-worker campaigns with complete provenance, explicit failure categories, and reproducibility hashes. Longer memory runs show that injected trial history can reduce repeated-bad proposals relative to no memory, while seed replicates show that small AUC changes should not be over-interpreted. The stronger conclusion is therefore methodological: autonomous experimentation agents should be evaluated not only by task metrics, but by governance metrics that expose whether the experimentation process itself was bounded, reproducible, and scientifically inspectable.

REFERENCES

- [1] Vivek Trivedy. 2026. The Anatomy of an Agent Harness. LangChain Blog. <https://blog.langchain.com/the-anatomy-of-an-agent-harness/> component that is not the model is the harness. Cited for the Agent = Model + Harness decomposition used throughout this paper..